```
{
safeFound = 0;
for(i=0;i<n;i++)
{
if(finish[i] == 0)
{
int safe = 1;
for(j=0;j<r;j++)
{
if(need[i][j] > avail[j])
{
safe=0;
break;
}
}
if(safe)
{
for(k=0;k<r;k++)
{
avail[k] = avail[k]+ alloc[i][k];
}
safe_sequence[count] = i;
finish[i] = 1;
safeFound = 1;
count++;
}
}
}
if(!safeFound)
{
printf("System is in unsafe state! May occur the deadlock \n");
return 0;
}
}
printf("Safe sequence is: ");
for(i=0;i<n;i++)
{
printf("P%d ",safe_sequence[i]);
}
return 0;
```

```c
int main() {
int n, r, i, j, k;
printf("Enter number of processes: ");
scanf("%d", &n);
printf("Enter number of resource types: ");
scanf("%d", &r);
int alloc[n][r], max[n][r], need[n][r], avail[r];
int finish[n], safe_sequence[n];
int count = 0, safeFound;
printf("Enter Allocation Matrix:\n");
for(i=0;i<n;i++)
{
for(j=0;j<n;j++)
{
scanf("%d", &alloc[i][j]);
}
}
printf("Enter Max Matrix:\n");
for(i=0;i<n;i++)
{
for(j=0;j<r;j++)
{
scanf("%d",&max[i][j]);
}
}
printf("Enter Available Resources:\n");
for(i=0;i<r;i++)
{
scanf("%d", &avail[i]);
}
for(i=0;i<n;i++)
{
finish[i] = 0;
}
for(i=0;i<n;i++)
{
for(j=0;j<r;j++)
{
need[i][j] = max[i][j] - alloc[i][j];
}
}
while(count < n)
{
safeFound = 0;
for(i=0;i<n;i++)
```

```
mo@mo-VirtualBox:~$ ./a.out
Enter number of processes: 3
Enter number of resource types: 2
Enter Allocation Matrix:
1 0
0 1
1 1
Enter Max Matrix:
2 1
1 2
3 3
Enter Available Resources:
1 1
mo@mo-VirtualBox:~$
```